

The Booking System, Explained Like You're Not a Developer

Think of this whole backend as **one big filing room** shared by two hotels - Centurion Hotel and Mum's Garden Resort. One room, one set of filing cabinets, but every single folder has a sticky note on it saying which hotel it belongs to. That sticky note is the `property_id`. That's really the whole trick that makes "one backend, two hotels" work - nothing more magical than that.

Let's walk through it like a story: a guest lands on the website, and we follow their booking all the way to check-out. Then we'll open each filing cabinet drawer (each database table) one at a time.

The big picture, in one paragraph

A guest picks dates on the website. The system checks how many rooms of that type are actually free (it does this math on the spot, it doesn't keep a running "3 rooms left" counter lying around). The guest fills in their details, and a **booking** is created - but it's not confirmed yet, it's just "on hold," like a reserved seat at a restaurant that will be given away if you don't show up in 30 minutes. The guest gets an M-Pesa prompt on their phone, pays the deposit, and *that payment* is what flips the booking from "on hold" to "confirmed." Behind the scenes, hotel staff have their own login where they can see all this happening, check guests in and out, manage rooms, and refund people. That's the whole system.

The guest's journey, step by step

1. Browsing. The guest opens the site for either hotel. The site asks the backend "what room types do you have?" and gets back a list - Deluxe Room, Standard Room, etc. - each with photos, a price, and a list of amenities. Nothing booking-related happens yet, this is just a catalogue.

2. Checking dates. The guest picks check-in and check-out dates and how many people. The backend does a quick bit of arithmetic: *"This room type has 8 units total. How many of those 8 are already tied up by bookings that overlap these dates?"* Whatever's left over is what's shown as available. Importantly - **there is no separate "availability" list stored anywhere**. It's recalculated fresh every single time someone asks, by looking at existing bookings. This avoids a classic bug where a stored "rooms left" number quietly drifts out of sync with reality.

3. Booking is created - but it's not a real reservation yet. The guest submits their details. A booking record is created with the status `pending_payment`. Think of this like a store clerk putting an item behind the counter for you - it's yours for the next 30 minutes, not longer. This hold is what stops two different guests from both being sent a payment prompt for the same last room.

4. The M-Pesa prompt. The guest's phone buzzes with the familiar Lipa na M-Pesa prompt. They enter their PIN. This doesn't instantly mark the booking as paid - it just tells the system "a payment attempt has started."

5. Waiting for confirmation. The website quietly checks every couple of seconds: *"has that payment gone through yet?"* Once Safaricom confirms the money actually landed, two things happen automatically: a **payment** record gets marked `completed`, and the **booking** flips from `pending_payment` to `confirmed`. If the guest cancels the M-Pesa prompt or it times out, the payment is marked `failed` and they can try again.

6. If nobody pays. If the guest just closes the tab and never pays, nothing bad happens to anyone else - after 30 minutes, that hold automatically expires, the booking status becomes `expired`, and that room becomes available to the next guest again. No human has to clean this up.

7. Getting there. Days later, the guest arrives. Front-desk staff find the booking (by name, phone, or the booking code), assign it to an actual physical room, and check them in. When they leave, staff check them out. The booking is a running record of that guest's whole stay, from browsing to departure.

8. If they need to cancel. Every booking has a cancellation deadline attached (e.g. "free cancellation up to 15 days before"), worked out automatically from the hotel's cancellation policy at the moment of booking. Cancel before that and you get money back per the policy; cancel after, and you don't - this is enforced by the system, not left to a manual judgment call.

Meanwhile, on the hotel's side

Hotel staff and managers log into a separate admin panel. What they can do depends on their role:

- **Staff** are the front desk. They can see bookings, check guests in/out, take walk-in bookings over the phone, and see who's inquired about events.
- **Admins** can do everything staff can, plus they manage the actual content - room types, prices, photos, the gallery, special offers, cancellation rules, and issuing refunds.
- **Superadmins** work for Straight Group (the company itself, above hotel level) and can see and manage *both* hotels - the only role allowed to cross that boundary.

A Centurion staff member's login is hard-wired to only ever see Centurion's data. They can't accidentally (or deliberately) pull up a Mum's Garden booking - the system blocks it at the door.

Now, the filing cabinets themselves - one drawer at a time

Here's every "drawer" (database table) in plain language: what it's for, and a everyday-life analogy.

properties - the two hotels themselves

The master list with exactly two entries: Centurion Hotel and Mum's Garden Resort. Holds the basics - name, contact details, address, currency, and the official check-in/check-out times. Every other drawer points back to one of these two rows.

Analogy: the two folders labeled "Centurion" and "Mum's Garden" that sit at the very front of the filing cabinet, which every other folder is filed behind.

room_types - the categories of room each hotel sells

"Deluxe Garden Room," "Standard Room," "City View Suite" - each with its own price, bed type, how many people it sleeps, how many physical units exist, and a free-form list of amenities (since Centurion and Mum's Garden don't sell the same features).

Analogy: the laminated room-type cards you'd see on a price list at reception.

room_type_images - the photos for each room type

Just the pictures, in order, with one marked as the "cover photo" shown first.

Analogy: the photo sleeve clipped behind each room-type card.

rooms - the actual physical rooms

Room 101, Room 102, etc. - real rooms with a status (active, under maintenance, out of service). This is mostly for front-desk use (assigning an actual key to a guest) - the booking math in Section 2 above doesn't depend on this list, it just needs the *count* of units from `room_types`.

Analogy: the physical room keys hanging on the board at reception.

cancellation_policies - the refund rules

Defines things like "100% refund if you cancel 15+ days out, 50% if 14 days out, nothing inside 48 hours." Also holds the deposit percentage required to hold a booking.

Analogy: the small print on the back of a hotel voucher.

rate_plans - temporary price changes

Optional overrides for specific date ranges - e.g. a midweek discount in August. If none applies, the system just falls back to the room type's normal price.

Analogy: a "sale sticker" temporarily placed over the regular price tag.

users - guest accounts and staff logins, in one list

Guests can optionally create an account (not required - booking without one is fully supported). Staff, admins, and superadmins are also rows in this same table, distinguished by their role and which hotel (if any) they're tied to.

Analogy: the guestbook and the staff ID-badge list, kept in the same cabinet but clearly labeled by role.

bookings - the reservation itself

The center of the whole system. One row per stay: who, which room type, which dates, how many people, what it costs, how much has been paid, and its current status (on hold, confirmed, checked in, checked out, cancelled, no-show, or expired). Every guest gets a short human-friendly code like `MG-4F82A1` to reference it by.

Analogy: the actual reservation slip pinned to the booking board.

booking_rooms - which physical room(s) got assigned

Links a booking to one or more actual rooms, for bookings that need more than one room, and lets front-desk assign the specific room only once the guest is arriving (not at the moment of booking).

Analogy: the sticky note added to a reservation slip once a specific room key gets pulled for it.

payments - every payment attempt

One row per M-Pesa attempt (or cash/future card payment): the amount, whether it was a deposit or full payment, its status, and the M-Pesa receipt number once it succeeds.

Analogy: the receipt drawer - every attempt gets a slip, whether it succeeded or not.

refunds - money given back

Tracks a refund from request through to actually being paid out, and who (which staff

member) processed it.

Analogy: a refund request form, stapled to the original receipt.

offers - promotions and discounts shown on the site

"20% off midweek stays," with its own dates and terms.

Analogy: the flyer taped to the reception desk about this month's special.

gallery_items - the photo gallery for the whole site

Organized by category - rooms, dining, events, exterior - separate from the room-type-specific photos.

Analogy: the photo album left out in the lobby.

inquiries - contact/event/group enquiry form submissions

When someone fills out "I want to book a wedding for 150 people" or just a general contact form, it lands here - not in **bookings**, since there's no room/date being reserved yet, just a conversation starting.

Analogy: the message slips left at reception for the events manager to call back.

notifications_log - a record of every email/SMS/WhatsApp message sent

Mostly booking confirmations for now. Built with a **whatsapp** option already included so a future WhatsApp assistant can plug into the exact same log without any rework.

Analogy: the "sent mail" tray - proof of what was sent to whom and when.

audit_log - a paper trail of who changed what

If an admin edits a room's price or a booking's status, it's recorded here: who did it, what changed. Purely for accountability, guests and the website never see this.

Analogy: the sign-in/sign-out ledger for anyone touching hotel records.

event_types - the "Meetings & Events" content cards

Things like "Corporate Board Meetings" or "Bridal Showers" shown on the events page. These are just promotional descriptions of *types* of events the hotel can host - not an actual calendar booking. Someone interested clicks through to the **inquiries** form, not a booking flow.

Analogy: the laminated cards in reception showing "here's what we can host for you."

media_assets - the one shared photo-upload system

Every image uploaded anywhere in the admin panel (room photos, gallery, offers, event cards) goes through this single system, so there's one consistent way files are stored and served, instead of four different upload mechanisms.

Analogy: one central photo-processing desk that every department sends their pictures to, instead of each department having its own camera shop.

The three things worth remembering above everything else

1. **Nothing is duplicated between the two hotels.** Same tables, same code, same rules - the only thing that differs is the `property_id` sticky note and the actual content (room names, prices, photos) inside it.
2. **Availability is always calculated live,** never stored as a stale number - so it can't drift out of sync with what's actually booked.
3. **A booking isn't real until it's paid.** Everything before that - browsing, picking dates, filling in the form - is provisional and expires automatically if nobody follows through.

Still waiting on the hotel's answers before this goes live

A few blanks in the data still need the client to fill in - these are business decisions, not technical ones:

- The exact size/bed type/price for each room, per hotel.
- One official check-in and check-out time per hotel (some source material had two different times listed).
- The cancellation deposit percentage - currently blank, so nobody should launch with a guessed number.
- Whether card payments (Visa/Mastercard) get switched on later - the system already has a place for it - it's just not wired up yet